



NOMBRE: MARTINEZ HUICAB CHRISTOPHER

MATRICULA: 67374

NOMBRE DEL PROFESOR: Sergio A. Noh Puch

NOMBRE DE LA MATERIA: Concurrencia Y Paralelismo

FECHA DE ENTREGA: 28/09/2026

**NOMBRE DE LA INSTITUCIÓN: UNIVERSIDAD
AUTÓNOMA DE CAMPECHE (CAMPUS V)**

**NOMBRE DE LA CARRERA: INGENIERÍA EN
TECNOLOGÍA DE SOFTWARE**

UAC

Universidad Autónoma
de Campeche

"Del Enigma sin Albas a Triángulos de Luz"



Explicacion de conceptos.....	1
Análisis del Código Ensamblador.....	1
Explicación de las instrucciones:.....	2
Resultados y Conclusión.....	2
Conclusión:.....	3

Explicacion de conceptos

¿Qué es una condición de carrera?

Es una situación que ocurre cuando múltiples hilos o procesos acceden y manipulan datos compartidos de forma concurrente, y el resultado final depende del orden en que se ejecuten las instrucciones. Si el orden cambia (debido al planificador del sistema operativo), el resultado puede variar o ser incorrecto.

¿Por qué puede producir resultados diferentes al ejecutar varias veces el programa?

Porque el sistema operativo alterna la ejecución de los hilos de manera no determinista. En la operación `global_counter++`, el procesador realiza tres pasos: leer el valor, incrementarlo y escribirlo. Si el hilo A lee el valor (20), pero antes de escribir (21) el hilo B lee el mismo valor (20) y escribe (19), se pierde la actualización de uno de ellos. Esto produce resultados inconsistentes en cada ejecución.

¿Qué es una sección crítica y qué parte del código corresponde a ella?

La sección crítica es la parte del código que accede a recursos compartidos y que debe ejecutarse de forma atómica (como una sola operación indivisible) para evitar condiciones de carrera.

En este código, la sección crítica es:

```
global_counter++;
```

```
global_counter--;
```

Estas líneas son las que deben protegerse con el mutex.

Análisis del Código Ensamblador

Para generar el ensamblador, ejecuta:

```
bash
```

```
gcc -S race_test.c
```

Esto generará un archivo `race_test.s`. Busca las instrucciones dentro de las funciones `thread_routine` y `thread_routine_two`. Aquí tienes un fragmento típico (en sintaxis AT&T, común en Linux):

```

assembly
# Dentro de thread_routine (incremento)
movl  global_counter(%rip), %eax # Carga el valor de global_counter en el registro eax
addl  $1, %eax                  # Suma 1 al registro eax
movl  %eax, global_counter(%rip) # Guarda el valor de eax de vuelta en global_counter

# Dentro de thread_routine_two (decremento)
movl  global_counter(%rip), %eax # Carga el valor de global_counter en el registro eax
subl  $1, %eax                  # Resta 1 al registro eax
movl  %eax, global_counter(%rip) # Guarda el valor de eax de vuelta en global_counter

```

Explicación de las instrucciones:

movl (Move Long): Se utiliza para mover datos de 32 bits. En este caso, mueve el valor de la variable `global_counter` desde la memoria al registro `%eax`, y luego mueve el resultado del registro de vuelta a la memoria. Es el paso de "leer" y "escribir".

addl (Add Long): Realiza una suma de 32 bits. Aquí suma `$1` (el valor inmediato 1) al contenido del registro `%eax`. Corresponde al incremento `++`.

subl (Subtract Long): Realiza una resta de 32 bits. Aquí resta `$1` al contenido del registro `%eax`. Corresponde al decremento `--`.

¿Por qué ocurre la condición de carrera a nivel de ensamblador?

Porque la operación `global_counter++` se descompone en tres instrucciones (`movl`, `addl`, `movl`). Si el hilo A ejecuta el primer `movl` y luego el sistema operativo cambia al hilo B, el hilo B también ejecuta su `movl` leyendo el valor antiguo. Ambos suman 1 a su copia local y luego escriben. El último en escribir sobrescribe al primero, perdiendo una actualización.

Resultados y Conclusión

Resultados esperados:

Sin sincronización: El valor final de `global_counter` probablemente no será 20. Podría ser 19, 21, 18, etc., dependiendo de cuántas actualizaciones se pierdan. Al ejecutarlo varias veces, obtendrás resultados distintos.

Con sincronización: El valor final siempre será 20. El mutex garantiza que las operaciones de incremento y decremento se realicen de forma atómica, evitando que se pisen entre sí.

Conclusión:

La condición de carrera es un problema crítico en la programación concurrente. Sin mecanismos de sincronización, el acceso concurrente a variables compartidas produce resultados impredecibles y erróneos. El uso de `pthread_mutex` permite definir secciones críticas, asegurando la exclusión mutua y garantizando la consistencia de los datos. A nivel de ensamblador, se observa que una simple operación de alto nivel (++) se traduce en múltiples instrucciones, lo que crea una ventana de tiempo donde otro hilo puede interferir, confirmando la necesidad de sincronización.